

JUSTIFICACIÓN PRINCIPIOS SOLID

EJERCICIO
Aseguradora

En el diseño de nuestro programa para crear el diccionario, aplicamos los 5 principios SOLID para garantizar que el código sea fácil de mantener, leer y no se rompa si necesitamos agregar más cosas en el futuro. A continuación, explicamos cómo se aplicó cada uno:

1. **S – Single Responsibility Principle (Principio de responsabilidad única)**

Aplicación en el código: Este principio se refleja en la clara separación de tareas a través de distintas clases. La clase NormalizadorTexto tiene el único propósito de limpiar las cadenas y manejar el error si el texto está vacío. Por su parte, la clase TablaHashBase solo se dedica a almacenar las palabras y manejar colisiones, mientras que las funciones dentro de AlgoritmosOrdenamiento se limitan exclusivamente a ordenar matemáticamente los datos. Gracias a esto, ninguna clase hace el trabajo de otra.

2. **O – Open/Closed Principle (Principio de abierto/cerrado)**

Aplicación en el código: Lo implementamos al pasar del punto de eliminar repetidos al de contar frecuencias. En lugar de modificar y alterar la clase original TablaHashBase para forzarla a contar las palabras, creamos una nueva clase llamada TablaFrecuencias que hereda de ella. De esta forma, el código de la clase base se queda "cerrado" a modificaciones (no corremos el riesgo de romper lo que ya funcionaba), pero "abierto" a extensión al sobrescribir los métodos insertar y obtener_solo_palabras.

3. **L – Liskov Substitution Principle (Principio de sustitución de Liskov)**

Aplicación en el código: Las clases cumplen este principio gracias a la herencia. Nuestra clase TablaFrecuencias respeta por completo la estructura de su padre TablaHashBase. Esto permite que, si en el bloque principal del programa decidiéramos usar una instancia de TablaHashBase en lugar de TablaFrecuencias y llamáramos al método obtener_solo_palabras(), el programa no marcaría ningún error de compatibilidad y nos regresaría una lista válida de la misma manera.

4. **I – Interface Segregation Principle (Principio de segregación de interfaces)**

Aplicación en el código: Aunque Python no tiene interfaces estrictas, aplicamos la idea central haciendo que nuestras clases tengan métodos muy precisos y directos. Por ejemplo, NormalizadorTexto solo expone el método normalizar. Esto

evita que las clases tengan que cargar con métodos innecesarios (como funciones de ordenamiento dentro de la tabla hash) que no corresponden a su naturaleza ni van a ocupar.

5. D - Dependency Inversion Principle (Principio de inversión de dependencias)

Aplicación en el código: Este principio se nota mucho en cómo conectamos los algoritmos con los datos. Las funciones `merge_sort` y `quick_sort` no dependen de nuestra clase `TablaFrecuencias` directamente para funcionar. Simplemente reciben una lista estándar de Python (abstracción) que nosotros extraemos antes. Si en el futuro la empresa decide cambiar la tabla hash por una base de datos distinta, la clase de `AlgoritmosOrdenamiento` seguirá funcionando sin requerir ninguna modificación en su código interno.

JUSTIFICACIÓN PRINCIPIOS SOLID

EJERCICIO
Diccionario

Para el desarrollo del diccionario, estructuramos nuestro código basándonos en los 5 principios SOLID solicitados en las instrucciones, asegurando que cada uno se aplicara en clases y métodos concretos:

1. **S – Principio de Responsabilidad Única (SRP):**

- Aplicamos este principio al dividir todo el problema en bloques pequeños y especializados. La clase NormalizadorTexto tiene un solo trabajo: limpiar cadenas de texto y manejar el TextoVacioError. Por su parte, la clase TablaHashBase solo se encarga de almacenar datos y manejar colisiones, mientras que las funciones merge_sort y quick_sort se dedican exclusivamente a ordenar. Si en el futuro queremos cambiar la forma en que se quitan los acentos, sabemos que solo debemos modificar el método normalizar(), sin tocar para nada la lógica de ordenamiento o almacenamiento.

2. **O – Principio Abierto/Cerrado (OCP):**

- Lo implementamos claramente al pasar del punto 2 al punto 4. En lugar de modificar nuestra clase original TablaHashBase para meterle a la fuerza el conteo de palabras, decidimos crear la clase hija TablaFrecuencias. Al heredar, dejamos la clase base "cerrada a modificaciones" (no rompimos el código que ya funcionaba), pero la dejamos "abierta a extensión", ya que sobrescribimos los métodos insertar() y obtener_solo_palabras() para adaptarlos a tuplas de frecuencias.

3. **L – Principio de Sustitución de Liskov (LSP):**

- Este principio se cumple gracias a cómo diseñamos nuestra herencia. La clase hija TablaFrecuencias es 100% capaz de sustituir a su padre TablaHashBase. Si en el flujo principal del programa decidimos instanciar la clase padre en lugar de la hija y mandamos a llamar a tabla.obtener_solo_palabras(), el programa seguirá funcionando perfectamente y nos devolverá la misma estructura esperada (una lista de palabras), sin causar un error de sintaxis ni alterar el comportamiento del resto del sistema.

4. **I – Principio de Segregación de Interfaces (ISP):**

- Aunque Python no utiliza "interfaces" estrictas como otros lenguajes, aplicamos la filosofía de este principio evitando clases "todoterreno" o infladas. Nuestro NormalizadorTexto no tiene métodos de ordenamiento que nunca va a usar, y la TablaHashBase no está forzada a implementar métodos de limpieza de texto. Cada parte del código expone solo los métodos que realmente necesita (como `_hash()` o `insertar()`), sin obligar a otras partes del programa a depender de funciones inútiles.

5. **D – Principio de Inversión de Dependencias (DIP):**

- Lo aplicamos al desacoplar la extracción de datos de su procesamiento, especialmente visible en los algoritmos de ordenamiento. `merge_sort` y `quick_sort` no dependen directamente de nuestra clase `TablaHashBase`. En su lugar, dependen de una abstracción de alto nivel: simples listas de Python. Nosotros extraemos la lista con `tabla.obtener_solo_palabras()` y se la pasamos a los algoritmos. Esto significa que mañana podríamos cambiar toda la lógica de la tabla hash por una base de datos real, y los algoritmos de ordenamiento seguirían funcionando sin tener que modificarles ni una sola línea de código.